

ז' באלול תש"ף

מס' שאלון - 530 27
באוגוסט 2020

סמסטר 2020ב

מס' מועד 88

20905 / 4

שאלון בחינת גמר
20905 - שפות תכנות

משך בחינה: 4 שעות

בשאלון זה 10 עמודים

מבנה הבחינה:

בבחינה 3 שאלות.
מבחן בית דרך מערכת המטלות
עליכם לענות על כל השאלות.
משקל השאלות מפורט בגוף השאלון.

שימו לב:
את תשובותיכם יש לרשום על דפים בכתב יד ברור,
לסרוק וליצור קובץ PDF יחיד
ולהעלותו תוך 4 שעות דרך מערכת המטלות
כתבו בכתב יד ברור ומסודר.

חומר עזר המותר בשימוש:
ספר הקורס "Essentials of Programming Languages".
מדריך הלמידה "שפות תכנות".

בהצלחה !!!

שאלה 1 (25 נקודות)

שאלה זו עוסקת בשדרוג של שפת "וישגור" (שפת PROC) המתוארת בספר הלימוד בפרק 3 בעמודים 74-81. שפה זו מאפשרת הגדרה והפעלה של פרוצדורות לא רקורסיביות.

כזכור שפת PROC ("וישגור") מאפשרת הגדרת שגרות בעלות פרמטר יחיד. אם רוצים להגדיר שגרה בעלת כמה פרמטרים יש צורך להשתמש בטכניקה שנקראת **currying** (תזכורת: הייתה שאלה בממ"ן בנושא Curried procedures). בטכניקה זו בונים שגרה שמקבלת כפרמטר יחיד את הארגומנט הראשון ומחזירה שגרה אחרת שמקבלת כפרמטר יחיד את הארגומנט השני, וכן הלאה. בכך מדמים הפעלה של שגרה בעלת כמה פרמטרים ע"י סדרה של הפעלות של שגרות בעלות פרמטר יחיד.

לדוגמה, ניתן לבנות בשפת PROC (המקורית) שגרה המבצעת את פעולת החיסור, שדורשת שני פרמטרים, באמצעות שימוש בטכניקת **currying**:

```
let p = proc (x)
  proc (y) -(x,y)
in ((p 10) 4)
```

בדוגמה זו, p היא פרוצדורה המקבלת פרמטר x ומחזירה פרוצדורה אחרת המקבלת פרמטר y (ושגם מכירה את x). בהפעלת הפרוצדורה p , נשלח ארגומנט של 10 עבור הפרמטר x , בתמורה מוחזרת פרוצדורה אחרת שמצפה לפרמטר וגם אותה מפעילים עם ארגומנט שערכו 4. ולכן ערכה הסופי של תוכנית זו הוא 6.

בשאלה זו, ברצוננו להטמיע בהגדרות שפת PROC את היכולת להגדרת פרוצדורות בעלות 2 פרמטרים שנבנות באמצעות **currying** (ועושות שימוש רק בפרוצדורות בעלות פרמטר יחיד).

עליכם להרחיב את שפת PROC עם ביטוי חדש שתחבירו נתון על-ידי הדקדוק הבא:

Expression ::= Curry (identifier , identifier) Expression

Curry-exp (id1 id2 body)

כאשר,

- id1 id2 הם פרמטרים עבור הפרוצדורה שתיבנה
- Body הוא ביטוי המתאר את גוף הפרוצדורה שתיבנה.

הסבר:

הפונקציה שתבנה ע"י ביטוי Curry תיבנה בטכניקה שתוארה בתחילת השאלה. שימו לב, לא יתקבל פתרון המגדיר בניית פונקציה עם 2 פרמטרים, וביטוי נוסף להפעלת פונקציה עם 2 פרמטרים. יש להשתמש רק ביכולות הקיימות בשפה של הגדרה והפעלה של פונקציות עם פרמטר יחיד.

להלן דוגמה להמחשת השימוש בביטוי החדש:

```
let p= Curry (x,y) ->(x,y)
in
((p 10) 4)
```

דוגמה זו זהה לדוגמה שתוארה בתחילת השאלה, ובמקום שהמתכנת בשפה יממש בעצמו currying, השפה עצמה תאפשר לו זאת.

א. (7 נקודות)

להלן נתונים ה-scanner וה-parser של שפת "וישגור". כתבו מהו השינוי הדרוש בדקדוק של השפה כך שיתמוך בהגדרות הביטויים החדשים.

ה-scanner:

```
(define the-lexical-spec
'((whitespace (whitespace) skip)
(comment ("% (arbno (not #\newline))) skip)
(identifier(letter (arbno (or letter digit "_" "-" "?"))
symbol)
(number (digit (arbno digit)) number)
(number ("-" digit (arbno digit)) number)
))
```

ה-parser:

```
(define the-grammar
'((program (expression) a-program)

(expression (number) const-exp)
(expression
("-" "(" expression "," expression ")")
diff-exp)

(expression
("zero?" "(" expression ")")
```

```
zero?-exp)
```

```
(expression
  ("if" expression "then" expression "else" expression)
if-exp)
```

```
(expression (identifier) var-exp)
```

```
(expression
  ("let" identifier "=" expression "in" expression)
let-exp)
```

```
(expression
  ("proc" "(" identifier ")" expression)
proc-exp)
```

```
(expression
  ("(" expression expression ")")
call-exp)))
```

ב. (18 נקודות)

להלן הפרוצדורה *value-of* של שפת "וישגור". ולאחריה פרוצדורת העזר *apply-procedure* כפי שמופיעה בספר הלימוד בעמוד 79. ממשו את התמיכה בתפעול של הביטוי החדש.

הערות:

- המנשק לעבודה עם סביבה הוא אותו מנשק כפי שהוגדר עבור שפת "וישגור".
- במסגרת הפתרון, ניתן לכתוב פונקציות עזר במקרה הצורך.

;;value-of : Exp x Env → ExpVal

```
(define value-of
  (lambda (exp env)
    (cases expression exp
      (const-exp (num) (num-val num))
      (var-exp (var) (apply-env env var))
      (diff-exp (exp1 exp2)
        (let ((val1 (value-of exp1 env))
              (val2 (value-of exp2 env)))
```

```

(let ((num1 (expval->num val1))
      (num2 (expval->num val2)))
  (num-val(- num1 num2))))
(zero?-exp (exp1)
  (let ((val1 (value-of exp1 env)))
    (let ((num1 (expval->num val1)))
      (if (zero? num1) (bool-val #t) (bool-val #f)))))
(if-exp (exp1 exp2 exp3)
  (let ((val1 (value-of exp1 env)))
    (if (expval->bool val1)
      (value-of exp2 env)
      (value-of exp3 env))))
(let-exp (var exp1 body)
  (let ((val1 (value-of exp1 env)))
    (value-of body (extend-env var val1 env))))

(proc-exp (var body)
  (proc-val (procedure var body env)))

(call-exp (rator rand)
  (let ((proc (expval->proc (value-of rator env)))
        (arg (value-of rand env)))
    (apply-procedure proc arg))))

;; apply-procedure : Proc * ExpVal ->ExpVal
;; Page: 79
(define apply-procedure
  (lambda (proc1 val)
    (cases proc proc1
      (procedure (var body saved-env)
        (value-of body (extend-env var val saved-env))))))

```

המשך הבחינה בעמוד הבא

שאלה 2 (50 נקודות)

בספר הלימוד בעמודים 113-120 מתוארת שפת "וירמוז" (IMPLICIT-REFS).

ברצוננו להרחיב שפה זו עם מספר ביטויים חדשים המאפשרים עבודה עם מילון (Dictionary) מילון הוא אוסף של זוגות key-value, כאשר לכל מפתח מותאם ערך. הערכים יכולים להיות מכל טיפוס הנתמך בשפה, וכן הערכים יכולים לחזור על עצמם בתוך המילון. המפתחות במילון זרים זה לזה, וגם הם יכולים להיות מכל טיפוס הנתמך בשפה.

להלן הדקדוק המגדיר את הביטויים החדשים שנדרש להוסיף:

1) $Expression ::= \{ \{ Expression : Expression \}^{(*)} \}$

dict-exp (keys values)

2) $Expression ::= <identifier>.add(Expression , Expression)$

dictAdd-exp (dict key value)

3) $Expression ::= \$Expression[Expression]$

dictKey-exp (dict key)

הסבר:

- ביטוי dict-exp מורכב מרשימת ביטויים המייצגים מפתחות (keys), ומרשימת ביטויים המייצגים ערכים (values) המשויכים לאותם מפתחות. ביטוי זה אחראי לבנייה של מילון עם מפתחות וערכים אלה, הערך המוחזר של ביטוי זה הוא המילון. (רמז: יש כאן טיפוס נתונים חדש)
- ביטוי dictAdd-exp מורכב משם מזהה (identifier) של מילון שכבר הוגדר קודם לכן, ממפתח (key) וערך (value). הביטוי צריך לשנות את ערכו של המילון הנתון, ולהוסיף אליו (תוך שמירה על התוכן הקיים) את המפתח והערך החדשים. ביטוי זה יחזיר ערך פיקטיבי של (num-val 27). בחישוב ביטוי זה נדרש לוודא כי המפתח החדש אינו קיים כבר במילון, במקרה כזה יש להחזיר שגיאה (בעזרת error: eopl).
- ביטוי dictKey-exp מורכב מביטוי dict שערכו הוא מילון, ומביטוי נוסף המשמש כמפתח (key). ביטוי זה אמור לחפש האם במילון קיים המפתח הנתון? אם כן, יוחזר הערך (value) המתאים לאותו מפתח. אם המפתח לא קיים במילון יש להחזיר שגיאה (בעזרת error: eopl).

להלן דוגמה לשימוש בביטויים החדשים:

```
> (run " let d= {}|
    in
      begin
        <d>.add(1:5);
        <d>.add(-(8,2): zero?(9));
        <d>.add(3: 12);
        -($d[3], $d[1])
      end")
(num-val 7)
>
```

בדוגמה זו, מוגדר בתחילה מילון ריק. לאחר מכן, מוסיפים לו 3 זוגות של מפתח-ערך, ולבסוף מוחזר ההפרש בין הערך המתאים למפתח שערכו 3 שהוא 12, לבין הערך המתאים למפתח שערכו 1 שהוא 5, ולכן תוצאת הביטוי היא: (num-val 7)

א. (15 נקודות – 5 נקודות לכל ביטוי)

להלן נתונים ה-scanner וה-parser של שפת "וירמוז". הסבירו וממשו את השינויים הדרושים בדקדוק של השפה כך שיתמוך בהגדרת הביטויים החדשים.

ה-scanner:

```
(define the-lexical-spec
'((whitespace (whitespace) skip)
  (comment ("% (arbno (not #\newline))) skip)
  (identifier(letter (arbno (or letter digit "_" "-" "?"))
                  symbol)
   (number (digit (arbno digit)) number)
   (number ("-" digit (arbno digit)) number)
  ))
```

ה-parser:

```
(define the-grammar
'((program (expression) a-program)
  (expression (number) const-exp)
  (expression ("-" "(" expression "," expression ")")
    diff-exp)
  (expression ("zero?" "(" expression ")") zero?-exp)
  (expression
```

```

("if" expression "then" expression "else" expression)
  if-exp)
(expression (identifier) var-exp)
(expression ("let" identifier "=" expression "in" expression)
  let-exp)
(expression ("proc" "(" identifier ")" expression)
  proc-exp)
(expression "(" expression expression ")") call-exp)
(expression
  ("letrec"
    (arbno identifier "(" identifier ")" "=" expression)
    "in" expression)
  letrec-exp)
(expression ("set" identifier "=" expression)
  assign-exp)))

```

ב. (35 נקודות – 15 נקודות לביטוי 1, 10 נקודות לביטוי 2, 10 נקודות לביטוי 3)

להלן הפרוצדורות *value-of* ו-*apply-procedure* של שפת "וירמוז".
 הסבירו וממשו במדויק את השינויים והתוספות הנדרשים לשילוב הביטויים החדשים בשפה זו.

הערות:

- המנשק לעבודה עם סביבה הוא אותו מנשק כפי שהוגדר עבור שפת "וירמוז".
- המנשק לעבודה עם החסן (store) הוא אותו מנשק כפי שהוגדר עבור שפת "וירמוז".
- במסגרת הפתרון, ניתן לכתוב פונקציות עזר במקרה הצורך.

```

;; value-of : Exp * Env ->ExpVal
;; Page: 118, 119
(define value-of
  (lambda (exp env)
    (cases expression exp
      (const-exp (num) (num-val num))

      (var-exp (var) (deref (apply-env env var)))

      (diff-exp (exp1 exp2)
        (let ((val1 (value-of exp1 env))
              (val2 (value-of exp2 env)))
          (let ((num1 (expval->num val1))
                (num2 (expval->num val2)))
            (diff num1 num2)))))))

```



```

        (num2 (expval->num val2)))
      (num-val
        (- num1 num2))))))

(zero?-exp (exp1)
  (let ((val1 (value-of exp1 env)))
    (let ((num1 (expval->num val1)))
      (if (zero? num1)
          (bool-val #t)
          (bool-val #f))))))

(if-exp (exp1 exp2 exp3)
  (let ((val1 (value-of exp1 env)))
    (if (expval->bool val1)
        (value-of exp2 env)
        (value-of exp3 env))))

(let-exp (var exp1 body)
  (let ((v1 (value-of exp1 env)))
    (value-of body
      (extend-env var (newref v1) env))))

(proc-exp (var body)
  (proc-val (procedure var body env)))

(call-exp (rator rand)
  (let ((proc (expval->proc (value-of rator env)))
        (arg (value-of rand env)))
    (apply-procedure proc arg)))

(letrec-exp (p-names b-vars p-bodies letrec-body)
  (value-of letrec-body
    (extend-env-rec* p-names b-vars p-bodies env)))

(assign-exp (var exp1)
  (begin
    (setref!
      (apply-env env var)

```

```
(value-of exp1 env))
(num-val 27))))))
```

```
(define apply-procedure
  (lambda (proc1 val)
    (cases proc proc1
      (procedure (var body saved-env)
        (value-of body
          (extend-env var (newref val) saved-env))))))
```

שאלה 3 (25 נקודות)

להלן נתונה תוכנית בשפת "ויקיש" (INFERRED):

```
let q= proc(y:?) zero? (- (y,10))
in
  let p= proc (x:?)
    if (x 5)
    then (q 10)
    else (q 20)
  in
    (p q)
```

ברצוננו לקבוע מהו טיפוס התוכנית (אם קיים). לשם כך, עליכם להשתמש באלגוריתם שנלמד בפרק 7 להקשת הטיפוס.

א. (10 נקודות)

הקצו לביטוי הנתון בשאלה ולתתי הביטויים שלו משתני-טיפוס (Type Variables) מתאימים, וחברו משוואות מתאימות לביטוי הנתון ולתתי הביטויים שלו.

ב. (15 נקודות)

פתרו את המשוואות שהרכבתם בסעיף א' תוך שימוש ב- substitution ו- unification בדומה למתואר בספר בעמודים 252-258. בסיום פתרון המשוואות רשמו מהו הטיפוס שאותו הקשתם עבור התוכנית הנתונה.

בהצלחה !